

A Design and Development of Web Application Vulnerability Detection Using Graph-based Modeling Analysis Over CAPEC

Teeramaet Bongkodmalee
School of ICT

Sirindhorn International Institute of
Technology, Thammasat University
Pathum Thani, Thailand
6322773761@g.siit.tu.ac.th

Supasake Vongvipaporn
School of ICT

Sirindhorn International Institute of
Technology, Thammasat University
Pathum Thani, Thailand
6322771294@g.siit.tu.ac.th

Preravitch Siripanich
School of ICT

Sirindhorn International Institute of
Technology, Thammasat University
Pathum Thani, Thailand
6322773761@g.siit.tu.ac.th

Somchart Fugkeaw
School of ICT

Sirindhorn International Institute of
Technology, Thammasat University
Pathum Thani, Thailand
somchart@siit.tu.ac.th

Chalee Vorakulpipat

Information Security Research Team
National Electronics and Computer
Technology Center
Pathum Thani, Thailand
Chalee.vorakulpipat@nectec.or.th

Abstract— This paper presents a new approach to web application vulnerability detection by integrating the Common Attack Pattern Enumeration and Classification (CAPEC) framework with our proposed graph analysis techniques. The proposed model aims at detecting the vulnerabilities of multiple web applications based on the examination of their attack patterns found in web server logs. We introduced a graph-based detection model to identify and classify attack patterns. This approach enhances the efficiency of the vulnerability discovery as well as improves the insightful analysis of the correlation between vulnerabilities and attack patterns. To demonstrate the functionality and efficiency of the proposed technique, we conducted the experiments to show how the system detects the vulnerabilities from the generated log data and provided the processing time of the system when it deals with multiple log files and attack patterns. The experimental results highlight the potential of our proposed system to advance web application security in the face of evolving cyber threats.

Keywords—Web Application Security, Vulnerability Detection, Graph-Based Analysis, Cyber Threats, Common Attack Pattern Enumeration and Classification (CAPEC), Log Analysis

I. INTRODUCTION

In the dynamic landscape of web applications, the persistent evolution of cyber threats necessitates innovative approaches to fortify digital defenses. Web application vulnerabilities, often exploited by adversaries with malicious intent, pose a substantial risk to the integrity and security of online systems. Traditional methods of detecting these vulnerabilities may fall short of addressing the sophisticated tactics employed by attackers. Most vulnerabilities detection approaches are usually done on the vulnerability assessment (VA) scanning tool or security incident and event management (SIEM) tool. Since cloud computing has been now employed to host a large number of web applications, such attack detection tools may not be applicable for implementing in cloud environment where there is higher scalability, extensive number of users and attackers, and the limited control of web server configuration. Basically, access and audit log files are instrumental for investigating and discovering attacks. However, dealing with a massive volume of log data generated by multiple web applications is non-trivial. The existing vulnerability discovery tools generally provide the list of results without the reveal of the attack

patterns of the detected vulnerabilities. In fact, the attack pattern of the vulnerabilities can be used for advanced threat analysis which will be beneficial for possible preventive actions. In addition, their detection performance is subject to the exhaustive scanning which is propositional to the number of data records in the log files.

The Common Attack Pattern Enumeration and Classification (CAPEC) [1] developed by the MITRE Corporation, offers a systematic and organized catalog of common attack patterns, providing standardized language for describing cyber threats. In our approach, CAPEC serves as a foundational database for creating attack signatures. Leveraging its extensive repository enhances the precision of our signature creation process, enabling our system to recognize and respond effectively to known attack patterns. The hierarchical classification within CAPEC facilitates a structured and nuanced analysis, contributing to the development of a graph-based model that captures the relationships between different attack patterns and their variations. Integrating CAPEC into our methodology strengthens the overall security posture of web applications by empowering our system to proactively detect and mitigate vulnerabilities.

This paper introduces a novel approach to web application vulnerability detection, leveraging CAPEC framework and employing graph analysis techniques. Specifically, our proposed system invoked web server access logs and error logs through the utilization of Python libraries. By generating structured information from CAPEC's extensive knowledge base of attack patterns, our approach establishes a comprehensive foundation for understanding potential threats. This structured information becomes the basis for a robust analysis of web server logs, enabling the identification and classification of attack patterns in a more nuanced and precise manner.

Key advantage of our approach is our newly proposed graph model, which provides a powerful mechanism for modeling complex relationships within the log files. The graph-based analysis offers a holistic view of the interdependencies among log entries, allowing for the detection of subtle patterns indicative of potential security breaches. The introduction of graph-based vulnerability pattern matching significantly enhances the efficiency of

searching within the logs, reducing implementation costs associated with traditional methods.

The results of this analysis are then visualized on a web interface, providing a user-friendly and accessible means for security practitioners to identify, investigate, and mitigate potential threats.

This paper contributes to the field by presenting a comprehensive solution that combines CAPEC analysis with graph techniques for enhanced web application vulnerability detection. The subsequent sections delve into the details of the methodology, including the simulation process, CAPEC-based information extraction, graph-based log analysis, and the development of the web interface. The effectiveness of the proposed approach is demonstrated through experimentation and evaluation, showcasing its potential to significantly advance web application security. As cyber threats continue to evolve, this research serves as a timely and vital contribution to the ongoing efforts to fortify digital ecosystems against malicious actors.

II. RELATED WORK

This section discusses some related works that provided solutions for detecting vulnerabilities in web application systems. For example, Liu et al. [2] proposed an approach aimed at identifying second-order security vulnerabilities on the web through the utilization of a web crawling method. The process involves the initial crawling of the website URL, transmission of anchor points, a subsequent crawl of URLs associated with the anchor points, and the subsequent identification of second-order web security vulnerabilities, particularly focusing on these flagged URLs. Experimental trials were conducted by the authors to demonstrate the effectiveness of the proposed method in detecting vulnerabilities such as SQL injection, XSS, and remote code execution.

In [3], Ge et al. presented the deployment of a detection system that integrates a crawler and Natural Language Processing (NLP). The crawler was designed to gather HTTP requests and responses to form a dataset. This dataset is classified based on parameter characteristics and the interaction of samples with a database. Subsequently, the text is transformed into word vectors, undergoes dimension reduction, and is serialized. The NLP algorithm was employed to train the dataset to enable the model capable of accurately predicting web vulnerabilities.

In [4], the authors introduced an Intrusion Detection System (IDS) designed for injection detection, specifically capable of identifying SQL injection attacks and Cross-Site Scripting. This system is constructed using SNORT as its foundation, leveraging its lightweight IDS capabilities. Notably, the system focuses on the mentioned attacks due to the challenge of developing a SNORT rule that achieves a low False Positive (FP) rate for other types of attacks.

In [5], Debnath et al. introduced a Real-time Log Analysis System model named LogLens. This system employs the Dynamic Programming Algorithm to identify abnormal log sequences associated with events or transactions. The algorithm systematically examines various components of a log entry, comparing essential parameters to predefined values, strings, patterns, or wildcards. This concept is integrated into the algorithm presented in this paper for the identification of SQL injections and cross-site scripting. The

algorithm utilizes collections of wildcards to compare patterns or signatures falling within each respective category. In [6], Zhang et al. introduced terms of variations in detection capabilities and addressed these in their paper by designing and implementing a web application vulnerability detection platform called Dirmap. This platform focuses on Script Code-based vulnerabilities with the specific goal of accurately and efficiently detecting information leakage in current web applications, filling the identified gap in existing detection capabilities.

In [7], Wirz et al. proposed a cloud-based Intrusion Detection System (IDS) design and implementation that leverages Apache Kafka and Spark Streaming for the classification and processing of high volumes of user inputs in REST HTTP communication. The system aims to detect anomalous inputs using a signature-based approach, constructing an IDS engine based on a set of known attack patterns. Later, in 2023, they proposed a scheme called OWADIS [8] as a cloud-based IDS with an emphasis on the detection of OWASP Top 10 Injection vulnerabilities, combined with additional common vulnerabilities such as brute-forcing and session hijacking.

In [9], the authors presented the empirical study of static code analysis tools designed for identifying vulnerabilities in Node.js code. To ensure a thorough tool evaluation, they compiled the most extensive curated dataset of Node.js code vulnerabilities. This dataset comprised 957 vulnerabilities, which were meticulously characterized and annotated by scrutinizing information found in npm advisory reports. This work represents the benchmark to help characterize existing Node.js code vulnerabilities.

In [10], The authors introduced a novel technique for detecting injection vulnerabilities in the server-side code of Java Web applications. This proposed method represents an integrated approach that harmoniously combines security slicing with hybrid constraint solving. The hybrid constraint solving mechanism coordinates automata-based solving with meta-heuristic search, providing a strategy for identifying and addressing injection vulnerabilities in the Java Web application's server-side code.

Recently, Chen et al. [11] proposed URadar, a method for detecting Unrestricted File Upload (UFU) and Unrestricted File Execution Upload (UEFU) vulnerabilities through an adaptive dynamic testing approach. URadar incorporates three fundamental designs: file upload interface identification, file type restriction inference, and invalid mutation combination filtration. These designs aim to effectively address two limitations observed in existing methods. To assess URadar's performance, the authors conducted extensive experiments, comparing it with state-of-the-art methods such as FUSE and RIPS.

Nevertheless, most solutions focus on detecting certain kinds of web application vulnerabilities or vulnerabilities in common web development languages. They did not consider the potential attacks that may occur in any web applications. Hence, their solutions have limited coverage for detecting the undefined patterns of various attacks. Also, they mostly overlooked the correlation of activities that might cause the subsequent attack. Lastly, the integration of comprehensive attack patterns discovery and detection performance has not been explicitly provided by existing works.

III. TECHNOLOGY BACKGROUND

A. CAPEC

CAPEC, the Common Attack Pattern Enumeration and Classification is a robust framework crafted by MITRE Corporation to systematically categorize and elucidate cyber-attack patterns. Employing a hierarchical taxonomy, it organizes attack methods into top-level categories with progressively detailed subcategories. This structured approach facilitates a granular classification of cyber threats. Operating in conjunction with cybersecurity standards such as common weakness enumeration (CWE) [12] and common vulnerability enumeration (CVE) [13], CAPEC, and OWASP 10 [14] complements the identification of software weaknesses by focusing on the specific technique attackers employ for exploitation. Its integration with threat modeling processes assists security professionals in identifying potential threats and vulnerabilities, informing the development of targeted security controls and countermeasures. Recognizing the dynamic nature of cyber threats, CAPEC is actively updated to describe emerging attack methodologies. It is commonly used as standardized and continually evolving reference for cybersecurity professionals engaged in understanding, classifying, and mitigating cyber threats. CAPEC is thus utilized to construct the tool facilitates the generation of mappings that link system preconditions, hierarchical attack patterns, and strategies for mitigating the impact of attacks on the targeted system. This underscores the practical application of CAPEC in system design and security enhancement.

B. Graph-based Model

Graphs are frequently used for handling huge structural information in different areas [15] such as pattern recognition, computer vision, distributed systems, cybersecurity, and bioinformatics, relying on the foundational principles of graph theory. Graph databases like Neo4j and Amazon Neptune facilitate efficient storage and retrieval of graph-structured data, while algorithms for subgraph isomorphism, often implemented in libraries like NetworkX, determine if a given graph contains a specific pattern. Graph neural network [16] was proposed to support the representation learning model and optimization. In cybersecurity, these methods contribute to the detection of attack patterns within network logs, enhancing anomaly detection and cybersecurity threat identification [17, 18].

C. Flask

Flask, an eminent and lightweight web application framework [19], is a cornerstone in the realm of web development, particularly for projects that necessitate a high degree of customization and flexibility. Originating from the Python ecosystem, Flask has been meticulously designed to provide developers with a solid, yet unobtrusive, foundation for web applications. Its minimalistic and modular approach allows for the seamless integration of various components, making it an ideal choice for projects where a tailored front-end experience is paramount.

At the heart of Flask's philosophy is the "micro-framework" concept. Unlike more heavyweight frameworks that prescribe specific tools and libraries, Flask offers the

freedom to choose the most suitable tools for a given project. This flexibility is particularly advantageous in the development of front-end interfaces, where the requirements can vary significantly from one project to another. Flask's compatibility with a wide range of libraries and extensions for tasks such as form validation, database integration, and user authentication, ensures that it can be adapted to the specific needs of any project.

IV. OUR PROPOSED MODEL

This section presents the details of our proposed model.

A. System Model

Fig. 1 exhibits our proposed system model.

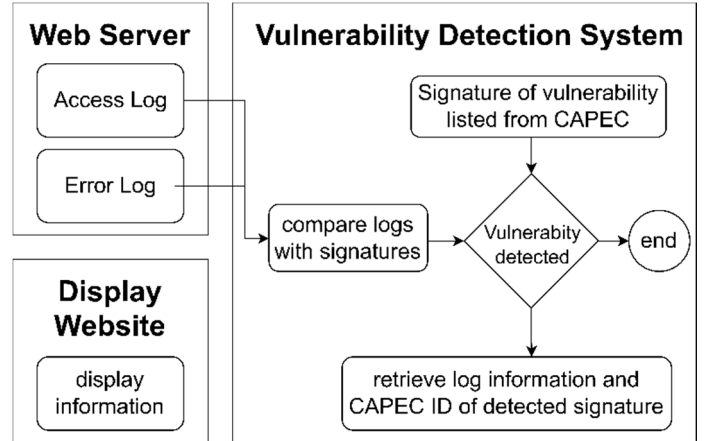


Fig. 1. Proposed system model

The proposed model for web application vulnerability detection as shown in Fig. 1 integrates the Common Attack Pattern Enumeration and Classification (CAPEC) framework with graph analysis techniques, aiming to improve efficiency and reduce implementation costs. The proposed model comprises the following system modules:

- **Web Server Logs Processor:** This system module invokes the web server access logs and error logs, utilizing Python libraries to aggregate and synthesize the log files for subsequent analysis.
- **Vulnerability Detection System:** This system module is constructed based on two major components (1) the CAPEC-Based Information Extraction responsible for extracting structured information from the CAPEC knowledge base enriches the understanding of potential threats. CAPEC's extensive attack pattern taxonomy enables the categorization of vulnerabilities within the simulated logs, and (2) Graph-Based Analysis used to formulate a graph to represent each log entry as a node, capturing relationships between entries as edges. This holistic representation facilitates the detection of subtle patterns indicative of security breaches. Graph-based vulnerability pattern matching enhances search efficiency, minimizing false positives and reducing computational resources.
- **Web Interface Visualization:** Results are presented through a user-friendly web interface. This interface allows security practitioners to interact with

visualized data, aiding in the identification, investigation, and prioritization of potential threats.

During the generation of structured attack pattern information, the project encounters potential threats and attacks emanating from the Common Attack Pattern Enumeration and Classification (CAPEC). Adversarial Tactics, Techniques, and Procedures (TTP) are systematically scrutinized and consolidated into a database, utilizing web access logs and error logs from the webserver to facilitate detection. Illustrative instances of such attacks encompass Cross-Site Scripting (XSS) and SQL injection. The database undergoes further refinement of threat details by extracting sub-techniques to augment the effectiveness of threat hunting. For each threat, the stored information encompasses the attack name, its associated child or sub-technique, and the corresponding signature or pattern.

The detection method implements a subgraph matching method for the detection system, where the graph stores attributes of each log in the node to create a directed graph. The steps of detection are as follows:

1. Parse the access logs and error logs and then create a unified log by utilizing the NetworkX libraries, establishing the foundation for constructing a provenance graph. This amalgamated log is used to compile comprehensive data from diverse logs.
2. Transform attack patterns, derived from the CAPEC, into a graph format for archival purposes.
3. Verify whether the attack pattern serves as a subgraph within the initially generated combined graph. If the attack pattern is recognized as a subgraph, that attack pattern is found in the system.

B. Graph-based matching Implementation

Graphs play a crucial role in detecting attack patterns, and there are two types employed: the log graph and the attack pattern graph. When transforming logs in, each log attribute is extracted to construct a graph. For access logs, attributes such as IP address, URL, timestamp, HTTP method, response code, response size, and query parameters are extracted. In error logs, the attributes include timestamp, message level, IP address, and error information. These extracted attributes serve as nodes in the graph, and the relationships between them are represented as edges. For instance, in the access log, the IP address is connected to the URL, and the URL is linked to other attributes, forming a network of nodes. Similarly, in the error log, the IP address is connected to the message level, and the message level is associated with other attributes. This graph-based representation enables a visual and structural understanding of the relationships between different log attributes, aiding in the identification and analysis of attack patterns as shown in Fig. 2.

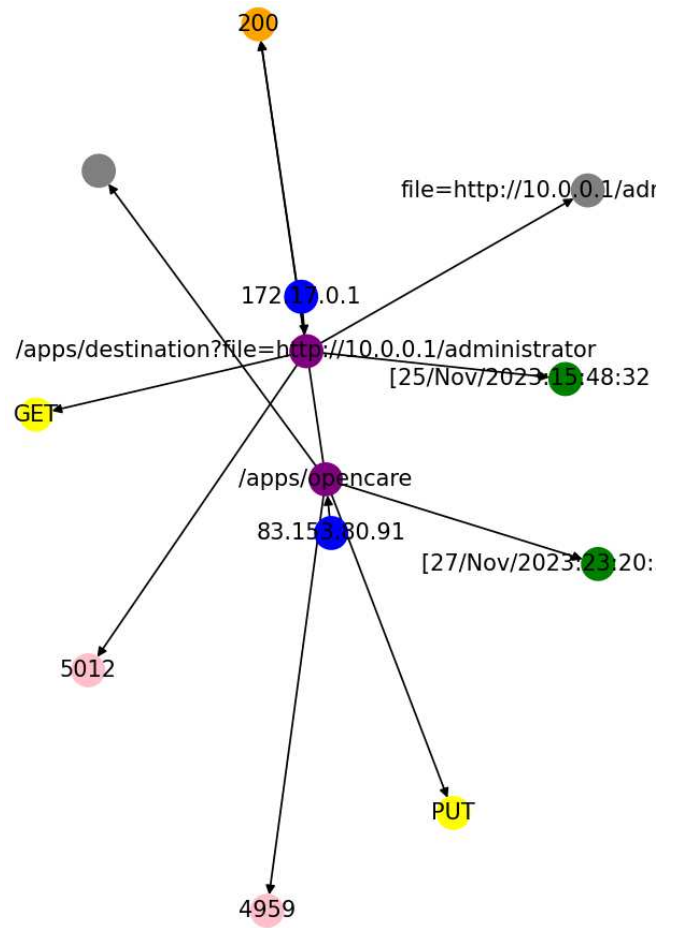


Fig. 2. Graph transformed from logs

The procedure involves transforming the attack patterns extracted from the CAPEC into a structured graph representation. For example, an attack pattern with the CAPEC ID 664, the nodes and edges are delineated as illustrated in the result below.

```
664: {
  'nodes': [
    ('error', {'type': 'Level_of_message'}),
    ('172.17.0.1', {'type': 'IP'}),
    ('File does not exist: http://10.0.0.1/admin', {'type': 'Error Info'}),
    ('File does not exist: http://10.0.0.1/admin_login', {'type': 'Error Info'}),
  ],
  'edges': [
    ('172.17.0.1', 'error'),
    ('error', 'File does not exist: http://10.0.0.1/admin'),
    ('error', 'File does not exist: http://10.0.0.1/admin_login'),
  ],
}
```

Fig. 3. below shows the procedure outlined in the CAPEC documentation which is converted into a node in the graph, and the relationships or dependencies between these steps are represented as edges.

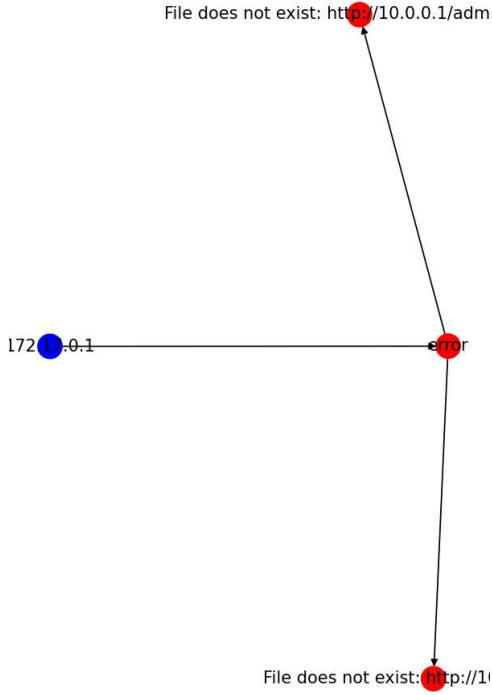


Fig. 3. Example of extracted attack pattern graph

In the detection method, Algorithm 1 is employed to ascertain the presence of an attack by determining whether the attack pattern graph exists within the log graph. The algorithm performs a check to establish if the attack pattern graph is a subgraph of the log graph. If the attack pattern graph is identified as a subgraph within the log graph, it is deemed as a detected attack. This approach relies on the concept that if the sequence and structure of the log entries match those of a known attack pattern, it signifies a potential security threat. The algorithm facilitates an automated and systematic means of identifying attacks by comparing the graph representations of the observed log data with predefined attack patterns.

ALGORITHM 1: Detecting Attack Pattern using Graph-Base Matching

```

1  FUNCTION find_matching_node(combined_graph,
2  pattern_graph, start_node):
3      start_node_attrs = pattern_graph.nodes[start_node]
4  IF start_node IN combined_graph.nodes AND
5  combined_graph.nodes[start_node]['type'] EQUALS
6  start_node_attrs['type']:
7      RETURN start_node
8  RETURN False
9  FUNCTION check_attack_pattern(combined_graph,
10 pattern_graph, start_node):
11  WHILE True:
12      start_node = find_matching_node(combined_graph,
13      pattern_graph, start_node)
14      IF NOT start_node:
15          RETURN False
16      successors_pattern =
17      LIST(pattern_graph.successors(start_node))
18      successors_combined =
19      LIST(combined_graph.successors(start_node))

```

```

20  FOR EACH next_node_pattern IN successors_pattern:
21      IF next_node_pattern NOT IN successors_combined:
22          RETURN False
23      IF LIST(pattern_graph.successors(next_node_pattern))
24      EQUALS []:
25          RETURN next_node_pattern
26      start_node = next_node_pattern
27  FUNCTION detect_attack_pattern_id(combined_graph,
28  attack_patterns):
29      matched_patterns = {}
30      FOR EACH pattern_id, pattern IN attack_patterns.items():
31          pattern_graph = pattern['graph']
32          FOR EACH node IN pattern_graph.nodes():
33              IF pattern_graph.in_degree(node) EQUALS 0:
34                  IF check_attack_pattern(combined_graph,
35                  pattern_graph, node):
36                      matched_patterns[pattern_id] = True
37                      BREAK
38      RETURN matched_patterns
39  matched_attack_patterns =
    detect_attack_pattern_id(combined_graph, attack_patterns)

```

C. Web application Display

Fig. 4. demonstrates how the web application's interface, developed using Flask, integrates with the Python-based attack detection code. This system employs a graph-based approach to identify attack patterns, each associated with a CAPEC ID number. Upon detection, this ID is utilized to query a database that holds details on various CAPEC Attack Patterns. The system then extracts and displays the specific mitigation strategies linked to the identified attack's CAPEC ID.

Potential Attack Patterns and Mitigations

CAPEC ID: 597

Mitigation:

- Design: Input validation. Assume that user inputs are malicious. Utilize strict type, character, and encoding enforcement.
- Design: Configure the access control correctly.
- Implementation: Use indirect references rather than actual file names.
- Design: Proxy communication to host, so that communications are terminated at the proxy, sanitizing the requests before forwarding to server host.
- Implementation: Use possible permissions on file access when developing and deploying web applications.
- Implementation: Perform input validation for all remote content, including remote and user-generated content.
- Design: Run server interfaces with a non-root account and/or utilize chroot jails or other configuration techniques to constrain privileges even if attacker gains some limited access to commands.

Fig. 4. Example of Web APP display

V. EXPERIMENT

We conducted experiments to evaluate the accuracy and effectiveness of how the model can detect known threats

from logs. The experiments were conducted on a computer with Intel Core i5-10300H, 4 P-Core, 8 threads, 1MB L2 Cache, and 8MB L3 Cache. As proof of concept, we did the simulation by generating the log files and employing a Python program, leveraging various libraries and techniques to replicate the characteristics of genuine web server logs. The generation of random values for parameters such as timestamps and IP addresses was facilitated by the utilization of the Faker library [20] in Python. This approach enhances the authenticity of the logs by introducing variability akin to real-world scenarios.

To further emulate a realistic log structure, we incorporated the DirBuster tool [21]—a penetration testing utility designed for the discovery of hidden directories and files on web servers. This tool allowed us to generate path URIs that align with common patterns found in genuine server logs, contributing to the overall fidelity of our simulated logs.

Additionally, to introduce a spectrum of log entries encompassing both benign and malicious activities, we integrated PayloadAllTheThings [22]. This tool facilitated the implementation of various exploits within the log entries, allowing us to evaluate and analyze the effectiveness of log analysis and intrusion detection systems under diverse conditions.

For instance, the provided log entries below are sample server logs and errors, displaying interactions with the server on a specific date and time.

```
'172.17.0.1 -- [25/Nov/2023:15:48:32 +0700] "GET
/apps/search?file=http://10.0.0.1/administrator HTTP/1.0" 200 5012
"https://www.watson-hughes.org/list/blogauthor.php" "Mozilla/5.0
(iPad; CPU iPad OS 10_3_4 like Mac OS X) AppleWebKit/533.1
(KHTML, like Gecko) FxiOS/18.3t3547.0 Mobile/650385 Safari/533.1"',
```

```
'172.17.0.1 -- [25/Nov/2023:15:48:32 +0700] "GET
/apps/search?file=http://10.0.0.1/administrator HTTP/1.0" 200 1201
"https://www.watson-hughes.org/list/blogauthor.php" "Mozilla/5.0
(iPad; CPU iPad OS 10_3_4 like Mac OS X) AppleWebKit/533.1
(KHTML, like Gecko) FxiOS/18.3t3547.0 Mobile/650385 Safari/533.1"',
```

```
'[Sun Nov 25 15:48:32 2023] [error] [client 172.17.0.1:50770] File does
not exist: http://10.0.0.1/admin',
```

```
'[Sun Nov 25 15:48:32 2023] [error] [client 172.17.0.1:50770] File does
not exist: http://10.0.0.1/admin_login',
```

In the first two entries, successful HTTP GET requests are recorded, both attempting to access the path `/apps/search?file=http://10.0.0.1/administrator` using the HTTP/1.0 protocol. The server responded with a status code of 200, indicating success, and the entries include details such as the response size, referrer URL, and user-agent information suggesting the use of an iPad. However, the server encountered errors in the subsequent entries, revealing attempts to access files that do not exist at the specified paths, namely `http://10.0.0.1/admin` and `http://10.0.0.1/admin_login`. These error messages, along with the associated client IP addresses and timestamps, provide insights into potential issues or security concerns, such as attempted access to non-existent files or directories. Analyzing such log entries is essential for monitoring server activity, identifying potential threats, and ensuring the security and proper functioning of the web server.

VI. EXPERIMENT RESULT

To evaluate the performance of the algorithm, we conducted tests to measure its efficiency in detecting attacks based on varying amounts of input log data. The objective was to assess how rapidly the model could identify potential threats as the volume of log entries increased. Additionally, scalability tests were conducted to gauge the system's ability to handle an expanding set of attack patterns.

Table I and Table II present the detection times in relation to the number of logs and the variety of attack patterns run in 5 tests, respectively. Table I illustrates that the detection time remains unaffected by the number of logs. Conversely, Table II indicates that the detection time fluctuates depending on the number of attack patterns, showing a scaling trend in direct proportion to the number of attack patterns.

TABLE I. DETECTION TIME BASED ON THE AMOUNT OF LOGS

Number of logs	Execution time (10^{-5} second)				
	Test 1	Test 2	Test 3	Test 4	Test 5
100	2.66	3.35	3.65	3.49	3.26
1,000	3.14	3.16	3.35	3.41	3.12
10,000	3.84	3.39	3.62	3.54	3.45
100,000	5.24	5.51	3.80	5.17	3.89

TABLE II. DETECTION TIME BASED ON THE NUMBER OF ATTACK PATTERNS

Number of logs	Execution time (10^{-5} second)				
	Test 1	Test 2	Test 3	Test 4	Test 5
100	2.66	3.35	3.65	3.49	3.26
1,000	3.14	3.16	3.35	3.41	3.12
10,000	3.84	3.39	3.62	3.54	3.45
100,000	5.24	5.51	3.80	5.17	3.89

VII. CONCLUSION

We introduced the web application vulnerability detection method based on graph-based analysis technique in conjunction with the CAPEC framework. The proposed model offers a more efficient and cost-effective approach to identifying potential threats. The experimental results validate the effectiveness of the model, showcasing its capability to detect a wide range of attack patterns with precision. This work not only provides a practical solution to a pressing cybersecurity challenge but also opens avenues for further research and development in this critical area. As cyber threats continue to evolve, such innovative approaches will be necessary in securing digital infrastructures. For future works, we will extend the capability of our proposed scheme to work over MITRE ATT&CK for modeling threat hunting.

ACKNOWLEDGMENT

This research project, supported by the National Science and Technology Development Agency under the agreement record for the "Young Scientist and Technologist Program" (reference number: MOU-CO-2562-11110-TH), and

REFERENCES

- [1] MITRE Corporation, "Common Attack Pattern Enumeration and Classification (CAPEC)," [Online]. Available: <https://capec.mitre.org/>. [Accessed: Nov. 27, 2023].
- [2] M. Liu and B. Wang, "A Web Second-Order Vulnerabilities Detection Method," in *IEEE Access*, vol. 6, pp. 70983-70988, 2018, doi: 10.1109/ACCESS.2018.2881070.
- [3] X. Ge and M. Yue, "Design and Implementation of System of the Web Vulnerability Detection Based on Crawler and Natural Language Processing," 2022 7th International Conference on Information and Network Technologies (ICINT), Okinawa, Japan, 2022, pp. 67-71, doi: 10.1109/ICINT55083.2022.00018.
- [4] H. Alnabulsi and R. Islam, "Protecting Code Injection Attacks in Intelligent Transportation System," 2019 18th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/13th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE), 2019, pp. 799-806.
- [5] B. Debnath et al., "LogLens: A Real Time Log Analysis System," 2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS), 2018, pp. 1052-1062, doi: 10.1109/ICDCS.2018.00105.
- [6] C. Zhang, X. Lin, J. Chen and Y. Geng, "Dirmap: Web Application Vulnerability Detection Platform Based on Script Code," 2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C), Hainan, China, 2021, pp. 148-151, doi: 10.1109/QRS-C55045.2021.00030.
- [7] L. Wirz, R. Tanthanathewin, A. Ketphet and S. Fugkeaw, "Design and Development of A Cloud-Based IDS using Apache Kafka and Spark Streaming," 2022 19th International Joint Conference on Computer Science and Software Engineering (JCSSE), Bangkok, Thailand, 2022, pp. 1-6, doi: 10.1109/JCSSE54890.2022.9836264.
- [8] L. Wirz, A. Ketphet, N. Chiewnawintawat, R. Tanthanathewin and S. Fugkeaw, "OWADIS: Rapid Discovery of OWASP10 Vulnerability based on Hybrid IDS," 2023 15th International Conference on Knowledge and Smart Technology (KST), Phuket, Thailand, 2023, pp. 1-6, doi: 10.1109/KST57286.2023.10086878.
- [9] T. Brito et al., "Study of JavaScript Static Analysis Tools for Vulnerability Detection in Node.js Packages," in *IEEE Transactions on Reliability*, vol. 72, no. 4, pp. 1324-1339, Dec. 2023, doi: 10.1109/TR.2023.3286301.
- [10] J. Thomé, L. K. Shar, D. Bianculli and L. Briand, "An Integrated Approach for Effective Injection Vulnerability Analysis of Web Applications Through Security Slicing and Hybrid Constraint Solving," in *IEEE Transactions on Software Engineering*, vol. 46, no. 2, pp. 163-195, 1 Feb. 2020, doi: 10.1109/TSE.2018.2844343.
- [11] Y. Chen, Y. Li, Z. Pan, Y. Lu, J. Chen and S. Ji, "URadar: Discovering Unrestricted File Upload Vulnerabilities via Adaptive Dynamic Testing," in *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 1251-1266, 2024, doi: 10.1109/TIFS.2023.3335885.
- [12] MITRE Corporation, "Common Weakness Enumeration," [Online]. Available: <https://cwe.mitre.org>.
- [13] MITRE Corporation, "Common Vulnerabilities and Exposures," [Online]. Available: <https://cve.mitre.org>.
- [14] Open Web Application Security Project (OWASP). "OWASP Top Ten." [Online]. Available: <https://owasp.org/Top10/>. [Accessed: Nov. 27, 2023].
- [15] V. S. Pawar and M. A. Zaveri, "Graph based pattern matching," 2011 Eighth International Conference on Fuzzy Systems and Knowledge Discovery (FSKD), Shanghai, China, 2011, pp. 1022-1026, doi: 10.1109/FSKD.2011.6019722.
- [16] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" Proc. of Seventh International Conference on Learning Representations (ICLR), May 2019.
- [17] Z. Li, J. Zeng, Y. Chen, and Z. Liang, "Attackg: Constructing technique knowledge graph from cyber threat intelligence reports," in Proc. of ESORICS, 2022.
- [18] Z. Cheng, R. Dai, L. Wang, Z. Yu, Q. Lv, Y. Wang, & D. Sun. (2023). GHunter: A Fast Subgraph Matching Method for Threat Hunting. In Proc. of the 2023 26th International Conference on Computer Supported Cooperative Work in Design (CSCWD) (pp. page range). IEEE. DOI: 10.1109/CSCWD57460.2023.10152818.
- [19] Pallets Projects, "Flask," [Online]. Available: <https://flask.palletsprojects.com/>. [Accessed: Nov. 27, 2023].
- [20] <https://faker.readthedocs.io/en/master/> [Accessed: Nov. 27, 2023].
- [21] "dirbuster | Kali Linux Tools," *Kali Linux*. <https://www.kali.org/tools/dirbuster/>
- [22] "Payloads All The Things," *swisskyrepo.github.io*. <https://swisskyrepo.github.io/PayloadsAllTheThings/>